Department of Computer Science and Engineering

KPR Institute of Engineering and Technology

B.E. – Computer Science and Engineering

Laboratory Manual

U21CS504

Web Technologies Laboratory

Regulation 2021

Nov / Dec 2026

List of Experiments

Expr. No.	Name of the Experiments
1.	Static web page design using HTML and CSS
2.	BMI Calculator using JavaScript
3.	User registration form validation using JavaScript
4.	Implementing web applications with API calls using Java Script
5.	Event registration form using PHP and MySQL
6.	Web application for food delivery system using PHP and MySQL
7.	Web application for leave management using Servlet and MySQL
8.	Web application for online shopping using Servlet and MySQL
9.	Mini Project:

Rubrics for Laboratory Experiments

Criteria & Points / Content	Strongly Acceptable	Acceptable	Considerable	Below Expectations	Unacceptable
	4	3	2	1	0
Understanding the Problem Statement	The procedure is explained and well designed the problem with appropriate analysis	Some programs are written accurately	Major errors in program writing	Programs are not written/wrong program written	Programs are not written/wrong program written
Completion of target in Lab	100% target has been completed	80% target has been completed	70% target has been completed	50% target has been completed	25% target has been completed
User Interface Design	Well designed problem with appropriate analysis	Designed the problem with analysis	Missing some experimental procedure with partial analysis	Missing major experimental details and analysis	Could not be able to explain the program
Development	Implement Optimal solution with appropriate results	80% of implementation has been done	65% of implementation has been done. Few procedures were explained	No proper explanation on implementation	Could not be able to explain the program
Innovation	Most of the innovations are applied clearly	Some of the innovations are applied undoubtedly	Few innovations are applied clearly	innovations applied	None of the innovations used

Course Focus: HTML, CSS, JavaScript, APIs, PHP, MySQL, Java Servlets, and Database Applications

EXERCISE 1

STATIC WEB PAGE DESIGN USING HTML AND CSS

1. AIM

To design and develop a responsive static web page using HTML5 and CSS3 by applying appropriate HTML elements, attributes, CSS selectors, box model properties, layout techniques, and styling principles.

2. OBJECTIVES

After completing this exercise, the student will be able to:

1. Design a structured web page using semantic HTML elements.
2. Apply HTML attributes to configure web page components.
3. Apply CSS styles using selectors and properties.
4. Implement the CSS box model for page layout and spacing.
5. Design a visually organized static web interface.
6.

3. PROBLEM STATEMENT

Design and develop a static web page for a university, organization, business, or service of your choice. The web page must include a header, navigation menu, main content section, relevant information cards or sections, images or visual content, and a footer.

4. REQUIREMENTS

The web page must contain the following components:

1. Header section with title and logo or branding.
2. Navigation menu with at least four links.
3. Main content section.
4. At least three information cards or content sections.
5. Images or suitable visual elements.
6. Buttons or hyperlinks.
7. Footer section.
8. Responsive layout using CSS.
9. Proper use of CSS selectors.
10. Appropriate spacing using the CSS box model.
11. SUGGESTED APPLICATION AREAS

Students may develop any one of the following:

1. University Department Website
2. College Event Website
3. Online Course Information Page
4. Travel Information Website
5. Restaurant Website
6. Hospital Information Website
7. Portfolio Website
8. Product Landing Page
9. Startup or Business Website
10. Sports Club Website
11. STEP-BY-STEP PROCEDURE

Step 1: Create the Project Folder

Create a project folder with the following structure:

```
web-page/
├── index.html
├── css/
│   └── style.css
└── images/
```

Step 2: Create the HTML Structure

Create an `index.html` file.

Define the basic HTML5 document structure using:

1. `<!DOCTYPE html>`
2. `<html>`
3. `<head>`
4. `<title>`
5. `<body>`

Step 3: Create the Header

Design a header section containing:

1. Website title.
2. Logo or branding.
3. Short introductory text.

Step 4: Create the Navigation Menu

Add navigation links such as:

1. Home
2. About
3. Services

4. Contact

Step 5: Create the Main Content

Organize the main content using suitable semantic elements such as:

1. `<main>`
2. `<section>`
3. `<article>`
4. `<div>`

Step 6: Add Content Cards

Create at least three cards containing:

1. Heading.
2. Description.
3. Image or icon.
4. Button or link.

Step 7: Apply CSS Styling

Create a separate CSS file and apply:

1. Font properties.
2. Background properties.
3. Text alignment.
4. Margin.
5. Padding.
6. Border.
7. Border radius.
8. Width and height.
9. Box shadow.

Step 8: Apply CSS Selectors

Use appropriate selectors such as:

1. Element selector.
2. Class selector.
3. ID selector.
4. Group selector.
5. Descendant selector.

Step 9: Implement the CSS Box Model

Apply:

1. Content.
2. Padding.
3. Border.
4. Margin.

Step 10: Test the Web Page

Verify:

1. All links are working.
2. Images are displayed correctly.
3. The layout is properly aligned.
4. Text is readable.
5. The page is visually consistent.
6. The page is responsive on different screen sizes.
7. EXPECTED OUTPUT

A well-structured and visually organized static web page developed using HTML5 and CSS3.

8. SUBMISSION REQUIREMENTS
9. `index.html`
10. CSS file
11. Images used in the application
12. Screenshot of the completed web page
13. Source code

EXERCISE 2

BMI CALCULATOR USING JAVASCRIPT

1. AIM

To design and develop an interactive BMI Calculator using HTML, CSS, and JavaScript by accepting user input, performing calculations, validating input, and dynamically displaying the result.

2. OBJECTIVES

After completing this exercise, the student will be able to:

1. Create interactive forms using HTML.
2. Retrieve input values using JavaScript.
3. Perform arithmetic operations using JavaScript.
4. Implement functions and conditional statements.
5. Dynamically update the web page using DOM manipulation.

3. PROBLEM STATEMENT

Develop a BMI Calculator that accepts a person's weight in kilograms and height in meters and calculates the Body Mass Index using JavaScript.

BMI is calculated using:

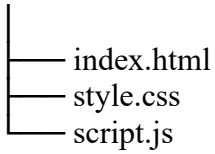
BMI = Weight / (Height × Height)

The application must display the calculated BMI and corresponding health category.

4. BMI CATEGORIES
5. Below 18.5 – Underweight
6. 18.5 to 24.9 – Normal Weight
7. 25.0 to 29.9 – Overweight
8. 30.0 and above – Obesity
9. STEP-BY-STEP PROCEDURE

Step 1: Create the Project Folder

bmi-calculator/



Step 2: Design the HTML Form

Create input fields for:

1. Weight in kilograms.
2. Height in meters.
3. Calculate button.
4. Reset button.
5. Result display area.

Step 3: Add Input Validation

Ensure that:

1. Weight is not empty.
2. Height is not empty.
3. Weight is greater than zero.
4. Height is greater than zero.
5. Only valid numeric values are accepted.

Step 4: Create the JavaScript Function

Create a function named `calculateBMI()`.

The function must:

1. Read the weight value.
2. Read the height value.
3. Calculate the BMI.
4. Round the result to two decimal places.
5. Determine the BMI category.
6. Display the result.

Step 5: Implement the BMI Calculation

Use the formula:

BMI = Weight / (Height × Height)

Step 6: Use Conditional Statements

Use conditional statements to determine the category:

if BMI < 18.5
Underweight

else if BMI < 25
Normal Weight

else if BMI < 30
Overweight

else
Obesity

Step 7: Display the Result Dynamically

Display:

1. BMI value.
2. Category.
3. Suitable message.

Step 8: Test the Application

Test the application using:

1. Valid weight and height.
2. Empty input.
3. Zero value.
4. Negative value.
5. Non-numeric input.
6. EXPECTED OUTPUT

The application must calculate and display the BMI value and corresponding category dynamically without reloading the page.

7. SUBMISSION REQUIREMENTS
8. HTML file.
9. CSS file.
10. JavaScript file.
11. Screenshots of valid and invalid test cases.
12. Source code.

EXERCISE 3

USER REGISTRATION FORM VALIDATION USING JAVASCRIPT

1. AIM

To design and develop a user registration form using HTML and JavaScript with client-side validation for name, email, password, mobile number, gender, and other user information.

2. OBJECTIVES

After completing this exercise, the student will be able to:

1. Design structured HTML forms.
2. Retrieve form values using JavaScript.
3. Validate user input using conditional statements.
4. Use regular expressions for input validation.
5. Display meaningful validation messages.

3. PROBLEM STATEMENT

Develop a user registration form for a web application. The form must collect user details and validate the entered information before submission.

4. REQUIRED FORM FIELDS

The form must include:

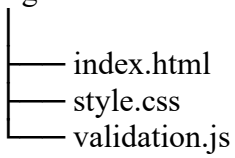
1. Full Name.
2. Email Address.
3. Mobile Number.
4. Password.
5. Confirm Password.
6. Gender.
7. Date of Birth.
8. Address.
9. Terms and Conditions checkbox.
10. Submit and Reset buttons.
11. VALIDATION REQUIREMENTS

The application must validate:

1. Name must not be empty.
2. Name must contain valid characters.
3. Email must follow a valid email format.
4. Mobile number must contain valid digits.
5. Password must satisfy the minimum length requirement.
6. Confirm Password must match Password.
7. Gender must be selected.
8. Terms and Conditions must be accepted.
9. Required fields must not be empty.
10. STEP-BY-STEP PROCEDURE

Step 1: Create the Project Folder

registration-form/



Step 2: Create the HTML Form

Create a form using:

1. `<form>`
2. `<label>`
3. `<input>`
4. `<select>`
5. `<textarea>`
6. `<button>`

Step 3: Assign Unique IDs

Assign unique IDs to all input fields.

Example:

1. `fullName`
2. `email`
3. `mobile`
4. `password`
5. `confirmPassword`

Step 4: Create the Validation Function

Create a JavaScript function named `validateForm()`.

Step 5: Read Input Values

Use JavaScript to retrieve the values entered by the user.

Step 6: Validate Required Fields

Check whether required fields contain valid values.

Step 7: Validate Email

Use a suitable regular expression to validate the email format.

Step 8: Validate Password

Check:

1. Minimum length.
2. Password and confirm password matching.

Step 9: Display Error Messages

Display appropriate messages near the corresponding input field.

Step 10: Display Success Message

If all validations are successful, display:

"Registration completed successfully."

Step 11: Test the Application

Test:

1. Empty form.
2. Invalid email.
3. Invalid mobile number.
4. Short password.
5. Password mismatch.
6. Valid registration data.
7. EXPECTED OUTPUT

The form must prevent submission when invalid data is entered and allow submission only when all required fields satisfy the validation rules.

8. SUBMISSION REQUIREMENTS
9. HTML file.
10. CSS file.
11. JavaScript validation file.
12. Screenshots of validation errors.
13. Screenshot of successful validation.
14. Source code.

EXERCISE 4

IMPLEMENTING WEB APPLICATIONS WITH API CALLS USING JAVASCRIPT

1. AIM

To develop an interactive web application that communicates with an external web API using JavaScript and displays dynamically retrieved data.

2. OBJECTIVES

After completing this exercise, the student will be able to:

1. Understand the purpose of web APIs.
2. Send requests from a JavaScript application.
3. Use the Fetch API.
4. Process JSON data.
5. Dynamically display API responses.

3. PROBLEM STATEMENT

Develop a web application that retrieves data from a suitable public API and displays the response dynamically on a web page.

4. SUGGESTED APPLICATIONS

Students may develop any one of the following:

1. Weather Information Application.
2. Country Information Application.
3. Currency Exchange Application.
4. News Information Application.
5. Public User Directory.
6. GitHub User Search Application.
7. Movie Search Application.
8. Random Quote Application.
9. APPLICATION REQUIREMENTS

The application must include:

1. User input or search field.
2. Search or submit button.
3. API request using JavaScript.
4. Response processing.
5. Dynamic display of results.
6. Loading message.
7. Error message.
8. Empty result handling.
9. STEP-BY-STEP PROCEDURE

Step 1: Select an API

Select a suitable public API.

Study:

1. API URL.
2. Required parameters.
3. Request method.
4. Response format.
5. JSON data structure.

Step 2: Create the Project Structure

api-application/

```
├── index.html
├── style.css
└── script.js
```

Step 3: Design the User Interface

Create:

1. Input field.
2. Search button.
3. Loading area.
4. Result display area.
5. Error message area.

Step 4: Capture User Input

Use JavaScript to retrieve the value entered by the user.

Step 5: Construct the API URL

Create the request URL using the user input.

Step 6: Send the API Request

Use the JavaScript Fetch API to send the request.

Step 7: Process the Response

Perform the following:

1. Check the response status.
2. Convert the response into JSON.
3. Extract required data.
4. Display the result.

Step 8: Handle Errors

Display suitable messages when:

1. The input is empty.
2. The API request fails.
3. The requested data is not found.
4. The server is unavailable.

Step 9: Display the Data

Present the retrieved data using:

1. Cards.
2. Tables.
3. Lists.
4. Structured information panels.

Step 10: Test the Application

Test:

1. Valid search.
2. Invalid search.
3. Empty search.
4. Network failure.
5. API error response.
6. EXPECTED OUTPUT

A dynamic web application that retrieves data from an API and displays the information without reloading the entire web page.

8. SUBMISSION REQUIREMENTS
9. HTML file.
10. CSS file.
11. JavaScript file.
12. API documentation reference.
13. Screenshots of successful and unsuccessful API requests.
14. Source code.

EXERCISE 5

EVENT REGISTRATION FORM USING PHP AND MYSQL

1. AIM

To design and develop a database-driven event registration application using HTML, PHP, and MySQL.

2. OBJECTIVES

After completing this exercise, the student will be able to:

1. Create HTML forms for user data collection.
2. Process form data using PHP.
3. Connect PHP with MySQL.
4. Insert registration data into a database.
5. Retrieve and display stored records.

3. PROBLEM STATEMENT

Develop an event registration application that allows users to register for an event by submitting their personal and event-related details.

4. REQUIRED FORM FIELDS

The registration form must include:

1. Participant Name.
2. Email Address.
3. Mobile Number.
4. Institution or Organization.
5. Event Name.
6. Event Category.
7. Number of Participants.
8. Registration Date.
9. DATABASE DESIGN

Create a database named:

event_management

Create a table named:

event_registrations

Suggested fields:

1. registration_id
2. participant_name
3. email
4. mobile
5. institution
6. event_name

7. category
8. participants
9. registration_date
10. STEP-BY-STEP PROCEDURE

Step 1: Set Up the Development Environment

Install and configure:

1. PHP.
2. MySQL.
3. Apache Server.
4. XAMPP, WAMP, or equivalent development environment.

Step 2: Create the Database

Create the database:

event_management

Step 3: Create the Table

Create the `event_registrations` table with suitable data types.

Step 4: Create the Registration Form

Create an HTML form containing all required fields.

Step 5: Configure the Form

Set:

1. Appropriate action.
2. POST method.

Step 6: Create the Database Connection

Create a PHP file such as:

`db.php`

Use PHP to establish a connection with MySQL.

Step 7: Process the Form Data

Create a PHP file to:

1. Receive submitted data.
2. Validate input.
3. Establish database connection.

4. Insert data into the database.

Step 8: Validate Input

Check:

1. Required fields.
2. Email format.
3. Mobile number.
4. Duplicate registration where applicable.

Step 9: Insert Data

Use an appropriate SQL INSERT operation.

Step 10: Display Confirmation

Display:

"Event registration completed successfully."

Step 11: Display Registered Participants

Create a page to retrieve and display registration details using SELECT.

Step 12: Test the Application

Test:

1. Valid registration.
2. Empty input.
3. Invalid email.
4. Duplicate registration.
5. Database connection failure.
6. EXPECTED OUTPUT

A working event registration application that stores and displays participant information using PHP and MySQL.

8. SUBMISSION REQUIREMENTS
9. HTML form.
10. PHP files.
11. Database creation script.
12. Screenshots.
13. Source code.

EXERCISE 6

WEB APPLICATION FOR FOOD DELIVERY SYSTEM USING PHP AND MYSQL

1. AIM

To design and develop a database-driven food delivery web application using PHP and MySQL that allows users to view food items, place orders, and manage order information.

2. OBJECTIVES

After completing this exercise, the student will be able to:

1. Design a multi-page web application.
2. Create and manage relational database tables.
3. Implement CRUD operations using PHP and MySQL.
4. Process user orders.
5. Display dynamic database content.

3. PROBLEM STATEMENT

Develop a food delivery web application that allows customers to browse food items, select items, provide delivery details, and place orders.

4. REQUIRED MODULES

The application must include:

1. User Registration.
2. User Login.
3. Food Menu.
4. Food Item Details.
5. Order Placement.
6. Customer Details.
7. Order Summary.
8. Order Status.
9. DATABASE DESIGN

Suggested database:

food_delivery

Suggested tables:

users

1. user_id
2. name
3. email
4. password
5. mobile
6. address

food_items

1. food_id
2. food_name
3. category
4. description
5. price
6. availability

orders

1. order_id
2. user_id
3. order_date
4. total_amount
5. order_status

order_items

1. order_item_id
2. order_id
3. food_id
4. quantity
5. price
6. STEP-BY-STEP PROCEDURE

Step 1: Create the Project Structure

food-delivery/

```
├── index.php
├── login.php
├── register.php
├── menu.php
├── order.php
├── db.php
├── css/
├── images/
└── admin/
```

Step 2: Create the Database

Create the `food_delivery` database.

Step 3: Create the Required Tables

Create tables for users, food items, orders, and order items.

Step 4: Implement User Registration

Create a registration form and store user details in the database.

Step 5: Implement User Login

Validate user credentials against the database.

Step 6: Display Food Items

Retrieve food items from the database and display them dynamically.

Each food item should display:

1. Food name.
2. Description.
3. Price.
4. Availability.
5. Order button.

Step 7: Implement Order Selection

Allow the user to:

1. Select food items.
2. Specify quantity.
3. Calculate total price.

Step 8: Capture Delivery Details

Accept:

1. Customer name.
2. Mobile number.
3. Delivery address.

Step 9: Store the Order

Insert order information into the database.

Step 10: Display Order Confirmation

Display:

1. Order ID.
2. Ordered items.
3. Quantity.
4. Total amount.
5. Delivery details.
6. Order status.

Step 11: Implement Administration Features

Provide suitable administrative functions to:

1. Add food items.
2. Update food items.
3. Delete food items.
4. View orders.
5. Update order status.

Step 12: Test the Application

Test:

1. User registration.
2. Login.
3. Food browsing.
4. Order placement.
5. Invalid input.
6. Database operations.
7. EXPECTED OUTPUT

A functional food delivery web application that allows users to browse food items and place orders while storing the information in a MySQL database.

8. SUBMISSION REQUIREMENTS
9. Complete PHP project.
10. Database script.
11. Screenshots of major modules.
12. Test cases.
13. Source code.
14. Project documentation.

EXERCISE 7

WEB APPLICATION FOR LEAVE MANAGEMENT USING SERVLET AND MYSQL

1. AIM

To design and develop a database-driven leave management web application using Java Servlets, HTML, JDBC, and MySQL.

2. OBJECTIVES

After completing this exercise, the student will be able to:

1. Develop interactive web applications using Servlets.
2. Process GET and POST form requests.
3. Establish database connectivity using JDBC.
4. Implement session management.
5. Perform database operations using SQL.

3. PROBLEM STATEMENT

Develop a leave management application that allows employees or students to apply for leave and enables an administrator or authorized user to view and update the leave status.

4. REQUIRED MODULES

The application must include:

1. User Login.
2. Leave Application.
3. Leave History.
4. Leave Approval.
5. Leave Rejection.
6. Leave Status Display.
7. Logout.
8. DATABASE DESIGN

Suggested database:

leave_management

users

1. user_id
2. username
3. password
4. role

leave_requests

1. leave_id
2. user_id
3. leave_type
4. from_date

5. to_date
6. reason
7. status
8. applied_date
9. STEP-BY-STEP PROCEDURE

Step 1: Set Up the Development Environment

Install and configure:

1. Java JDK.
2. Apache Tomcat.
3. MySQL.
4. JDBC Driver.
5. IDE such as Eclipse, IntelliJ IDEA, or NetBeans.

Step 2: Create the Database

Create the database:

leave_management

Step 3: Create Database Tables

Create tables for users and leave requests.

Step 4: Create the Web Application

Create a Java Web Application project.

Organize the project into:

1. HTML pages.
2. Servlet classes.
3. Database utility classes.
4. CSS files.

Step 5: Implement Login

Create a login form.

The servlet must:

1. Receive username and password.
2. Connect to MySQL using JDBC.
3. Validate credentials.
4. Create an HTTP session.
5. Redirect the user to the appropriate page.

Step 6: Implement Leave Application

Create a leave application form containing:

1. Leave type.
2. From date.
3. To date.
4. Reason.

Step 7: Process Leave Request

The servlet must:

1. Retrieve form data.
2. Validate the input.
3. Retrieve the logged-in user from the session.
4. Insert the leave request into the database.
5. Set the initial status as "Pending".

Step 8: Display Leave History

Retrieve the user's leave records from the database and display:

1. Leave type.
2. From date.
3. To date.
4. Reason.
5. Status.

Step 9: Implement Approval and Rejection

Authorized users must be able to:

1. View pending requests.
2. Approve requests.
3. Reject requests.
4. Update the database status.

Step 10: Implement Logout

Invalidate the session using:

```
session.invalidate()
```

Step 11: Test the Application

Test:

1. Valid login.
2. Invalid login.
3. Leave application.
4. Leave history.

5. Approval.
6. Rejection.
7. Logout.
8. EXPECTED OUTPUT

A working leave management web application that allows users to apply for leave and authorized users to process leave requests.

8. SUBMISSION REQUIREMENTS
9. Java Servlet source files.
10. HTML files.
11. JDBC database connection code.
12. SQL database script.
13. Screenshots.
14. Test cases.
15. Project documentation.

EXERCISE 8

WEB APPLICATION FOR ONLINE SHOPPING USING SERVLET AND MYSQL

1. AIM

To design and develop an online shopping web application using Java Servlets, JDBC, MySQL, HTML, CSS, and session management.

2. OBJECTIVES

After completing this exercise, the student will be able to:

1. Develop a complete database-driven web application.
2. Implement user authentication using Servlets.
3. Retrieve and display products from a MySQL database.
4. Manage shopping cart data using sessions.
5. Implement product and order database operations.
6.

3. PROBLEM STATEMENT

Develop an online shopping web application that allows users to register, log in, browse products, add products to a shopping cart, place orders, and view order details.

4. REQUIRED MODULES

The application must include:

1. User Registration.
2. User Login.
3. Product Listing.
4. Product Search.
5. Product Details.
6. Shopping Cart.
7. Order Placement.
8. Order History.
9. Logout.
10. DATABASE DESIGN

Suggested database:

online_shopping

users

1. user_id
2. name
3. email
4. password
5. mobile
6. address

products

1. product_id
2. product_name
3. category
4. description
5. price
6. stock
7. image

orders

1. order_id
2. user_id
3. order_date
4. total_amount
5. status

order_items

1. order_item_id
2. order_id
3. product_id
4. quantity
5. price
6. STEP-BY-STEP PROCEDURE

Step 1: Set Up the Development Environment

Install:

1. Java JDK.
2. Apache Tomcat.
3. MySQL.
4. JDBC Driver.
5. Java IDE.

Step 2: Create the Database

Create the database:

online_shopping

Step 3: Create Database Tables

Create the tables:

1. Users.
2. Products.
3. Orders.

4. Order Items.

Step 4: Implement User Registration

Create a registration form to accept:

1. Name.
2. Email.
3. Password.
4. Mobile number.
5. Address.

Insert the data into the users table.

Step 5: Implement User Login

The login servlet must:

1. Receive login credentials.
2. Validate them using JDBC.
3. Create a session after successful login.
4. Redirect the user to the product page.

Step 6: Display Products

Retrieve products from the database and display:

1. Product name.
2. Category.
3. Price.
4. Stock availability.
5. Add to Cart button.

Step 7: Implement Product Search

Allow users to search products based on:

1. Product name.
2. Category.

Step 8: Implement Shopping Cart

Use an HTTP session to store selected products.

The cart must display:

1. Product name.
2. Quantity.
3. Price.
4. Subtotal.

5. Total amount.

Step 9: Implement Order Placement

When the user confirms the order:

1. Create an order record.
2. Store order items.
3. Calculate total amount.
4. Update product stock.
5. Display order confirmation.

Step 10: Implement Order History

Retrieve previous orders of the logged-in user and display:

1. Order ID.
2. Order date.
3. Total amount.
4. Order status.

Step 11: Implement Administrative Functions

The administrator may:

1. Add products.
2. Update product details.
3. Delete products.
4. View orders.
5. Update order status.

Step 12: Implement Logout

Invalidate the user session and redirect the user to the login page.

Step 13: Test the Application

Test:

1. Registration.
2. Login.
3. Invalid login.
4. Product listing.
5. Product search.
6. Add to cart.
7. Update cart quantity.
8. Remove cart item.
9. Order placement.
10. Logout.
11. EXPECTED OUTPUT

A functional online shopping web application that supports user registration, login, product browsing, shopping cart management, order placement, and order history using Java Servlets and MySQL.

8. SUBMISSION REQUIREMENTS
9. Complete Java Web Application.
10. Servlet source files.
11. HTML and CSS files.
12. JDBC database connection code.
13. SQL database script.
14. Screenshots.
15. Test cases.
16. Project documentation.

GENERAL VIVA QUESTIONS

1. What is the purpose of HTML in a web application?
2. What is the difference between HTML and CSS?
3. What is the CSS box model?
4. What are CSS selectors?
5. What is JavaScript?
6. What is DOM manipulation?
7. What is client-side validation?
8. What is an API?
9. What is JSON?
10. What is the Fetch API?
11. What is PHP?
12. What is MySQL?
13. What is CRUD?
14. What is a primary key?
15. What is a foreign key?
16. What is a PHP session?
17. What is a Java Servlet?
18. What is a servlet container?
19. Explain the servlet life cycle.
20. What is JDBC?
21. What is the purpose of a `Connection` object?
22. What is a `PreparedStatement`?
23. What is an HTTP session?
24. What is a cookie?
25. Why is input validation important?
26. Why should database queries be parameterized?
27. What is the difference between GET and POST?
28. What is the role of a web server?
29. What is the difference between static and dynamic web pages?
30. Why is database normalization important?